AI Assisted Coding

Assignment 6.3

Name: M.PRASHANTH

Hall ticket no: 2303A51270

Batch no: 19

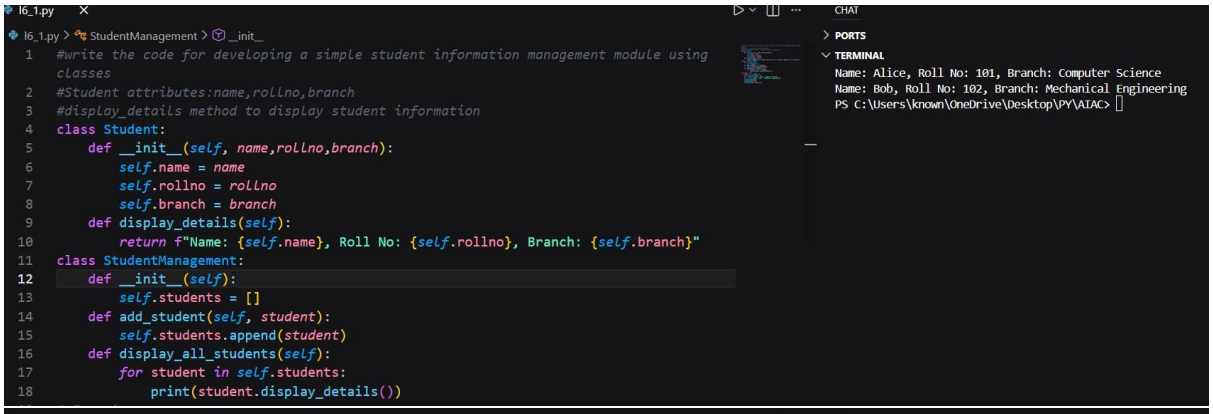
Task 1: Classes (Student Class) Prompt:

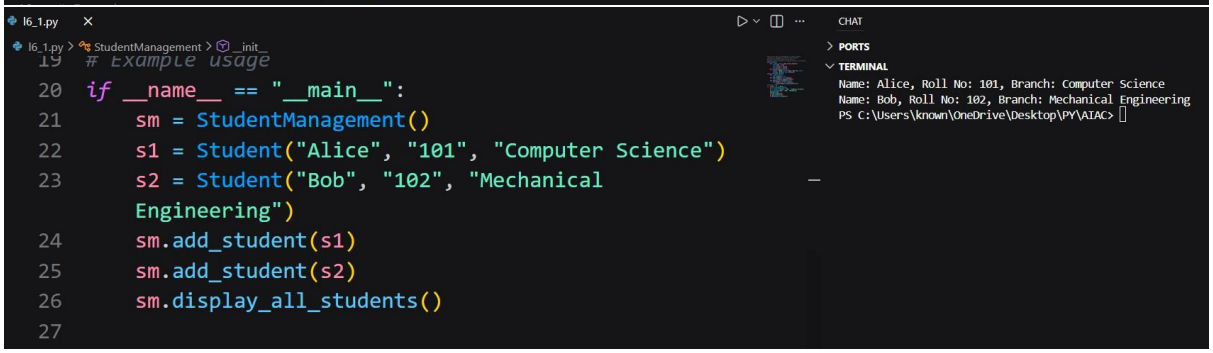
#Write the code for developing a simple student information management module using classes

#Student attributes:name,rollno,branch

#display_details method to display student information **Code**

& Output:





Explanation:

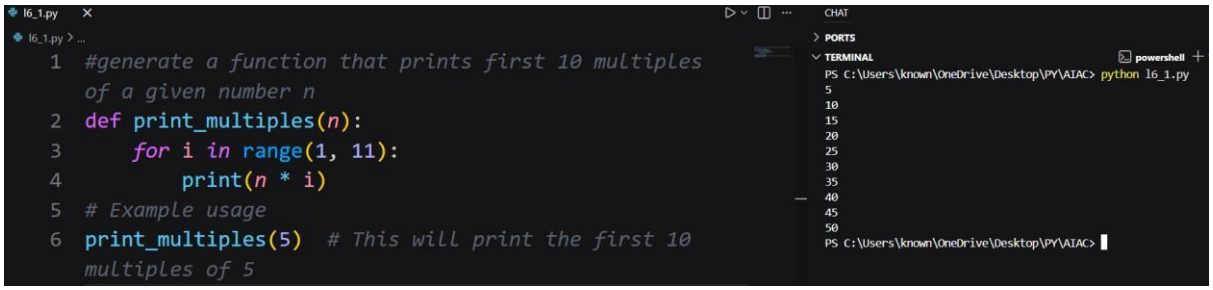
The AI-generated code correctly defines a Student class using object-oriented principles. The constructor initializes student attributes, and the display_details() method prints them clearly. The class structure is simple, readable, and functions correctly when an object is created and executed.

Task 2: Loops (Multiples of a Number)

Prompt:

Generate a function that prints first 10 multiples of a given number n **Code**

& Output:



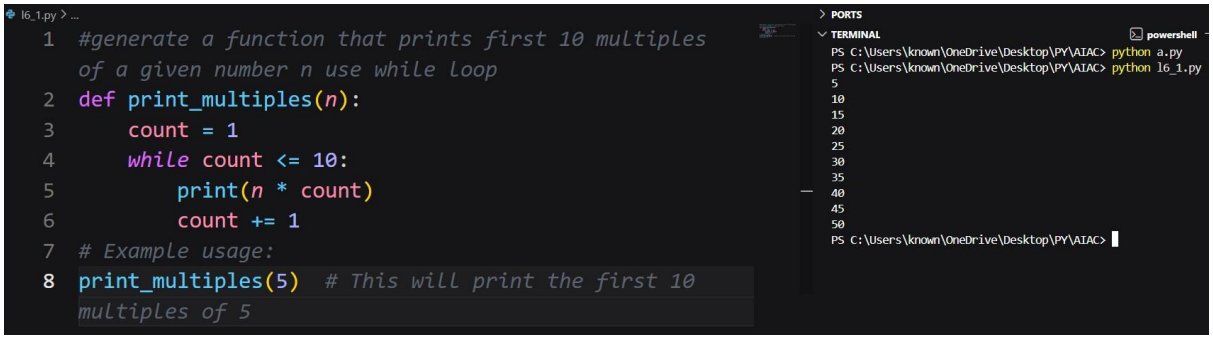
Explanation:

The AI-generated function uses a for loop to iterate from 1 to 10 and prints the multiples of the given number. In each iteration, the loop variable is multiplied by the input number. The loop boundaries are correctly defined, and the logic produces accurate results. This implementation is efficient and readable, making it ideal for tasks with a fixed number of iterations.

Prompt (Alternative Loop):

Generate a function that prints first 10 multiples of a given number n using while loop **Code**

& Output:



Explanation:

The while-loop version produces the same output as the for-loop version. While loops require manual control of the counter variable, making the for loop slightly cleaner and more readable for fixed iterations.

Task 3: Conditional Statements (Age Classification) Prompt:

Generate nested conditional statements to classify age groups.

Code & Output:

```python
#generate nested conditional statements to classify age groups
age = int(input("Enter your age: "))
if age < 0:
    print("Invalid age")
elif age <= 12:
    print("Child")
elif age <= 19:
    print("Teenager")
elif age <= 64:
    print("Adult")
else:
    print("Senior")
```

```
PS C:\Users\known\OneDrive\Desktop\PY\AIAC> python 16_1.py
Enter your age: 42
Adult
PS C:\Users\known\OneDrive\Desktop\PY\AIAC>
```

Explanation:

The AI-generated function uses nested if-elif-else conditions to classify age groups. Each condition checks a specific age range in increasing order. The structure ensures that only one category is returned for a given age. The logic is clear, correct, and easy to verify, making the code understandable for beginners and suitable for real-world classification tasks.

Prompt (Alternative Logic):

Generate a program to classify age groups using alternative if-elif-else statements **Code**

& Output:

```python
#generate a program to classify age groups using alternative if-elif-else statements
age = int(input("Enter your age: "))
if age < 13:
    print("You are a child.")
elif 13 <= age < 20:
    print("You are a teenager.")
elif 20 <= age < 65:
    print("You are an adult.")
else:
    print("You are a senior citizen.")
```

```
PS C:\Users\known\OneDrive\Desktop\PY\AIAC> python 16_1.py
Enter your age: 42
Adult
PS C:\Users\known\OneDrive\Desktop\PY\AIAC> python 16_1.py
Enter your age: 12
You are a child.
PS C:\Users\known\OneDrive\Desktop\PY\AIAC>
```

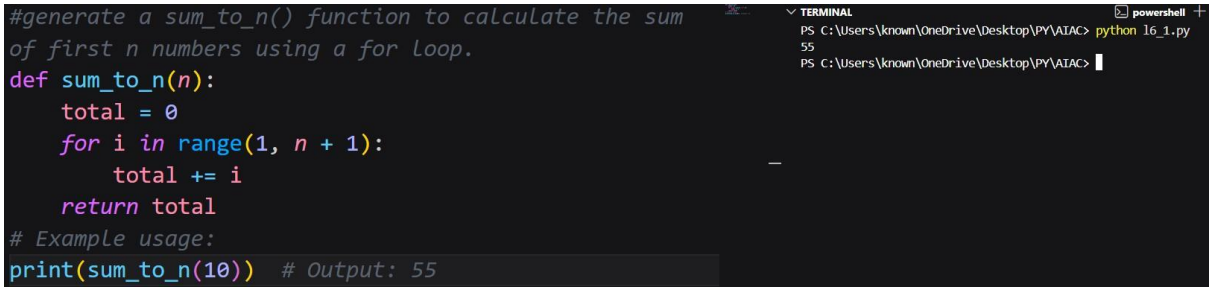
Explanation:

This alternative approach uses a dictionary and a loop to determine the age group. While this method is flexible and scalable, it is more complex than the if-elif-else approach. For simple classification problems, the original conditional structure is more readable and easier to maintain.

Task 4: For and While Loops (Sum of First n Numbers) Prompt:

Generate a Python function to calculate the sum of first n natural numbers using a for loop.

Code & Output:



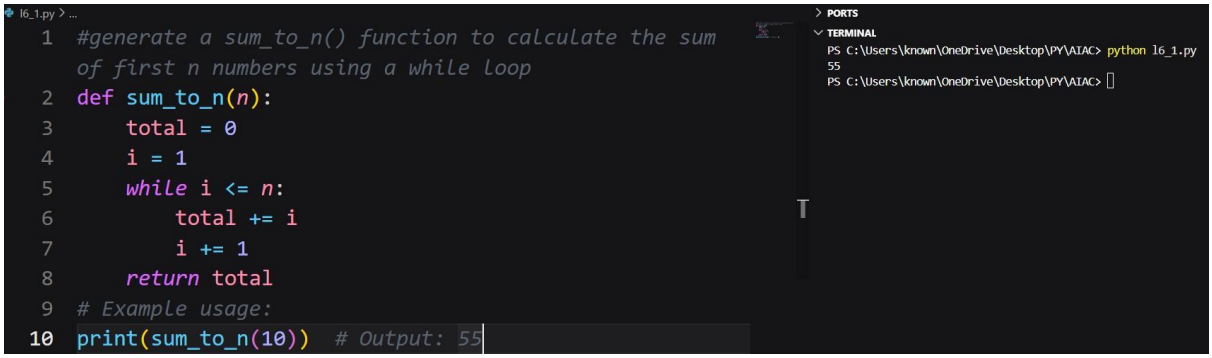
Explanation:

The AI-generated function calculates the sum by iterating through numbers from 1 to n and adding them to a total variable. The loop logic is correct and produces accurate results. This approach is easy to understand and works efficiently for small to moderate values of n.

Prompt (Alternative Loop):

Generate a sum_to_n() function to calculate the sum of first n numbers using a while loop

Code & Output:



Explanation:

The while-loop version produces the same output as the for-loop version. While loops require manual control of the counter variable, making the for loop slightly cleaner and more readable for fixed iterations.

Task 5: Classes (Bank Account Class)

Prompt:

Generate a Bank Account class with methods such as deposit(), withdraw(),and check_balance() with meaningful comments.

Code & Output:

```python
#generate a Bank Account class with methods such as deposit(), withdraw(), and check_balance() with meaningful comments.
class BankAccount:
    def __init__(self, account_holder, initial_balance=0):
        """
        Initialize a new bank account instance.

        :param account_holder: Name of the account holder
        :param initial_balance: Starting balance of the account (default is 0)
        """
        self.account_holder = account_holder
        self.balance = initial_balance

    def deposit(self, amount):
        """
        Deposit a specified amount into the bank account.

        :param amount: Amount to be deposited (must be positive)
        """
```

```
PS C:\Users\known\OneDrive\Desktop\PY\AIAC> python 16_1.py
Current balance: $1000.00
Deposited: $500.00. New balance: $1500.00
Withdrew: $200.00. New balance: $1300.00
Insufficient funds for this withdrawal.
Current balance: $1300.00
PS C:\Users\known\OneDrive\Desktop\PY\AIAC>
```

```python
class BankAccount:
    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"Deposited: ${amount:.2f}. New balance: ${self.balance:.2f}")
        else:
            print("Deposit amount must be positive.")

    def withdraw(self, amount):
        """
        Withdraw a specified amount from the bank account.

        :param amount: Amount to be withdrawn (must be positive and less than or equal to the current balance)
        """
        if amount > 0:
            if amount <= self.balance:
                self.balance -= amount
                print(f"Withdrew: ${amount:.2f}. New balance: ${self.
```

```
PS C:\Users\known\OneDrive\Desktop\PY\AIAC> python 16_1.py
Current balance: $1000.00
Deposited: $500.00. New balance: $1500.00
Withdrew: $200.00. New balance: $1300.00
Insufficient funds for this withdrawal.
Current balance: $1300.00
PS C:\Users\known\OneDrive\Desktop\PY\AIAC>
```

```python
class BankAccount:
    def withdraw(self, amount):
            else:
                print("Insufficient funds for this withdrawal.")
        else:
            print("Withdrawal amount must be positive.")

    def check_balance(self):
        """
        Check and return the current balance of the bank account.

        :return: Current balance
        """
        print(f"Current balance: ${self.balance:.2f}")
        return self.balance
# Example usage:
if __name__ == "__main__":
    # Create a new bank account for John Doe with an initial balance of $1000
    account = BankAccount("John Doe", 1000)
```

```
PS C:\Users\known\OneDrive\Desktop\PY\AIAC> python 16_1.py
Current balance: $1000.00
Deposited: $500.00. New balance: $1500.00
Withdrew: $200.00. New balance: $1300.00
Insufficient funds for this withdrawal.
Current balance: $1300.00
PS C:\Users\known\OneDrive\Desktop\PY\AIAC>
```

```python
    # Check the initial balance
    account.check_balance()

    # Deposit $500 into the account
    account.deposit(500)

    # Withdraw $200 from the account
    account.withdraw(200)

    # Attempt to withdraw more than the current balance
    account.withdraw(2000)

    # Check the final balance
    account.check_balance()
```

```
PS C:\Users\known\OneDrive\Desktop\PY\AIAC> python 16_1.py
Current balance: $1000.00
Deposited: $500.00. New balance: $1500.00
Withdrew: $200.00. New balance: $1300.00
Insufficient funds for this withdrawal.
Current balance: $1300.00
PS C:\Users\known\OneDrive\Desktop\PY\AIAC>
```

Explanation:

The AI-generated Bank Account class demonstrates effective use of object-oriented programming. The constructor initializes the balance, and the methods allow depositing, withdrawing, and checking the balance. Conditional logic prevents withdrawal when the balance is insufficient. The code is clear, logically sound, and easy to extend, making it suitable for a basic banking application.